

A Certified Primal–Dual Solver for the Selective Routing Problem with Synchronisation: a Falsification Study of Strengthening Techniques and a Cut-Augmented Lagrangian Extension

David Chouchena

Tomer Tunitsky

Tel-Aviv University

Exact Algorithms for Combinatorial Optimization Problems

August 2026

Abstract

The Selective Routing Problem with Synchronisation (SRPS) asks which of a set of profitable jobs to serve, how to route parallel processors through them, and when to start each job, subject to the requirement that every processor a job needs must begin serving it at the same instant. We describe a certified primal–dual solver for SRPS pairing an Adaptive Large Neighbourhood Search with a Lagrangian relaxation of the joint-service coupling, so that every returned solution carries a valid per-instance upper bound and hence a certificate of proximity to optimality. On the 660-instance benchmark it attains a mean certified gap of 0.133%, a median of 0.046%, and proves optimality on 42.9% of instances.

We also develop a cut-augmented Lagrangian relaxation $L(\mu, \gamma, \nu)$ that inserts capacity and incompatibility inequalities in the projected selection space while preserving the per-processor orienteering decomposition. In standalone comparison on a 54-instance stratified sample, this tightens the certified bound against a matched-budget plain Lagrangian baseline, winning on 30 of 54 instances with a mean gap of 0.361% versus 0.455% (-0.094 pp). In a pipeline experiment with deliberately weakened initial bounds, the cut-augmented arm achieves a mean certified gap of 0.428% against 0.533% for the plain Lagrangian arm, with 12 instance wins to 8. This is the one dual-side technique in the study that demonstrably tightens the bound, and its benefit is conditional on the initial upper bound having genuine headroom.

The bulk of this report is a systematic attempt to strengthen that solver using ten techniques from the exact-algorithms syllabus, and an account of why nine fail. Two were the primary brief: importing the cut separation logic of the exact branch-price-and-cut method, and coupling the search more tightly to the Lagrangian dual. Both are falsified on structural rather than parametric grounds. Separation against the incumbent can never find a violated cut, because the incumbent is feasible by construction; redirected at the relaxed solution, where violations demonstrably exist, the natural feasibility cut is *unsound* on the majority of instances tested. The dual-guided operators are degenerate under their own initialisation—the fair profit split forces the reduced profit to be identically zero for every job—and once repaired, an isolating control attributes the apparent gain entirely to a warm-start dual solve, which a full-budget re-run then shows to be a screening artifact.

A ceiling argument computed from the solver’s own gap distribution bounds *any* technique aimed at the certified gap at 0.1136 *pp* on average and 0.0087 *pp* at the median—below the run-to-run variance of the experiments built to detect it—and every gap-directed technique tested was absorbed by it. The two aimed at runtime were not so bounded, and one delivered: parallel evaluation of the per-processor subproblems is $1.42\times$ faster with bit-identical bounds. Beyond the negative results we extract two methodological lessons: mechanism *activation* is not evidence of mechanism *effect*, and a verdict measured at one computational budget can reverse at another—in both directions.

Contents

1	Introduction	3
2	The Selective Routing Problem with Synchronisation	4
2.1	Binding constraints	4
2.2	Compact formulation	5
2.3	Benchmark	5
3	Related work	6
3.1	SRPS and its exact treatment	6
3.2	Metaheuristics for selective routing	6
3.3	Lagrangian relaxation and decomposition	6
3.4	Techniques this study draws on	6
4	Summary of techniques evaluated	7
5	The solver	7
5.1	Representation and feasibility	7
5.2	Destroy and repair operators	8
5.3	Synchronisation-aware insertion	8
5.4	Acceptance, adaptation, and multi-start	8
5.5	The Lagrangian bound	8
5.6	Subgradient scheme	9
5.7	The adaptive control loop	9
5.8	Certification	9
6	Baseline performance	10
6.1	Certified gaps	10
6.2	Certified optima and subproblem cost	10
6.3	Relation to the exact method	10
6.4	Headroom: a ceiling on any gap-directed improvement	10
7	Cut-augmented Lagrangian relaxation	10
7.1	Formulation	11
7.1.1	Capacity cuts	11
7.1.2	Incompatibility cuts	11
7.2	Experimental protocols	12
7.3	Standalone results	12
7.4	Pipeline results	13
7.4.1	Tight-UB protocol: the ceiling applies	13
7.4.2	Headroom protocol	13
7.5	Verdict	14
8	Study design	14
8.1	Notation	14
8.2	Instance subsets	14
8.3	Budget regimes	14
8.4	How each technique was tested and calibrated	16

9	Techniques targeting the dual bound	17
9.1	BPC-style cut separation	17
9.1.1	First design and its structural impossibility	17
9.1.2	The activation sweep and mechanism inertness	17
9.1.3	Redirecting separation at the relaxed solution	18
9.1.4	Why the natural cut is unsound	18
9.1.5	Verdict	19
9.2	Can the incumbent itself generate cuts?	19
9.3	Primal-to-dual seeding and dual stabilisation	20
9.4	Does the trust region survive a longer budget?	20
9.5	Alternative ways to combine the subproblem solutions	22
9.6	Warm-start dual effort scheduling	23
10	Techniques targeting the primal incumbent	23
10.1	Dual-guided destroy and repair	23
10.1.1	Design and the initialisation degeneracy	23
10.1.2	The original results are what the defect predicts	23
10.1.3	Correcting the design	24
10.1.4	Corrected results	24
10.1.5	Production-budget validation	24
11	Techniques targeting runtime	25
11.1	Reduced-cost variable fixing	25
11.2	Parallel subproblem evaluation	26
12	An uncoupled exact reference	26
13	Lagrangian decomposition: why the natural split cannot help	27
14	Methodological lessons	28
15	Threats to validity	29
16	Conclusions	29
A	Reproduction and results	30

1 Introduction

The Selective Routing Problem with Synchronisation (SRPS) arises from configuring the EMIR spectrograph at the Gran Telescopio Canarias [1]. The instrument has 55 pairs of sliding bars in parallel bands; observing a target requires moving one or more bar pairs into position, and a target lying between bands requires several bars positioned *simultaneously*. Observing time is scarce, so only a subset of requested targets can be served. The problem is therefore selective (which targets), routing (in what order each bar pair visits them), and scheduling (when each observation starts), with a synchronisation constraint binding the three together.

This report has two parts. The first describes a certified primal–dual solver: an Adaptive Large Neighbourhood Search supplies incumbents, a Lagrangian relaxation supplies valid upper bounds, and every solution is returned with a certificate of how far from optimal it can possibly be. The second, and longer, asks whether that solver can be strengthened using techniques from the exact-algorithms syllabus.

Two directions motivated the study:

Cut separation. The exact method for SRPS derives part of its strength from cutting planes separated over the synchronisation structure. If those inequalities are valid, they should be exploitable inside a heuristic — to filter infeasible candidate moves, or to tighten the dual bound.

Dual coupling. The Lagrangian multipliers $\mu_{j,k}$ carry information about which jobs the relaxation considers under- or over-penalised. Steering the destroy and repair operators by the reduced profit $b_j - \sum_k \mu_{j,k}$ should focus the search where the dual says value remains.

Both are falsified. Exhausting the surrounding design space added eight further techniques — primal-to-dual seeding, dual stabilisation, Lagrangian decomposition, warm-start effort scheduling, reduced-cost variable fixing, parallel subproblem evaluation, and a standalone exact reference — of which one improves what it targets. Beyond the ten-technique study, a cut-augmented Lagrangian extension (Section 7) is the one dual-side contribution that demonstrably tightens the bound. Sections 9–11 are organised by the metric each technique aims at, because that division is the organising fact of the study: everything aimed at the certified gap was bounded in advance by a ceiling computable from the solver’s own results, and the two aimed at runtime were not.

Contributions.

1. A certified primal–dual solver for SRPS (Section 5) and its evaluation on the 660-instance benchmark (Section 6).
2. A ceiling argument bounding *any* gap-directed strengthening, derived from the solver’s own gap distribution (Section 6.4).
3. A structural impossibility result for cut separation against a primal incumbent, and a soundness refutation of the natural feasibility cut when separation is redirected at the relaxed solution (Section 9.1).
4. A degeneracy proof for the dual-guided operator design, a corrected re-test, and the isolating control that overturns its apparent success (Section 10.1).
5. A proof that Lagrangian decomposition of the natural split cannot improve the bound, closing that cell analytically rather than leaving it untested (Section 13).
6. Two methodological guards — gate on effect rather than activation, and validate screen finalists at full budget — that between them would have detected every failure here in minutes rather than hours (Section 14).
7. A cut-augmented Lagrangian relaxation $L(\mu, \gamma, \nu)$ (Section 7) incorporating capacity and incompatibility inequalities in the projected selection space while preserving the per-processor decomposition; evaluated across three experimental protocols (standalone budget comparison, tight-bound pipeline, headroom pipeline) establishing when and why it tightens the certified bound.

2 The Selective Routing Problem with Synchronisation

2.1 Binding constraints

An instance provides a job set $J = \{1, \dots, n\}$, where job j yields profit b_j and requires a subset $K_j \subseteq K$ of the processor set K ; a transition-time matrix $(t_{i\ell})$ covering travel and processing; and a global horizon L . For each processor k , let $J_k \subseteq J$ denote the jobs requiring it. A solution selects a subset of jobs, routes each processor through its assigned jobs, and assigns start times. Three constraints bind:

1. **Horizon.** Elapsed time from the start depot to the end depot may not exceed L on any processor.
2. **Synchronisation.** All processors in K_j must start job j at the same instant s_j . For every arc $(i \rightarrow \ell)$ traversed by any processor, $s_\ell - s_i \geq t_{i\ell}$. Processors may idle before a synchronised start, and *that idle time counts toward L* .
3. **Consistency.** If any processor serves j , every processor in K_j must serve j . Partial service is forbidden.

The objective is to maximise $\sum_{j \in J} b_j y_j$, where y_j indicates selection.

Constraints 2 and 3 are distinct requirements and are easily conflated: consistency says *which* processors must serve a job, synchronisation says *when* they must do so. A solution can satisfy either without the other.

Constraint 2 is a difference-constraint system: earliest start times are longest paths in the arc-weighted graph induced by the routes, and a cyclic graph means no feasible schedule exists. This makes feasibility checkable in linear time and is the check the solver uses (Section 5.1). The idle-time clause separates “route length $\leq L$ ” from “makespan $\leq L$ ”: routes that each fit comfortably inside the horizon may still be jointly infeasible once synchronised waiting is charged. Section 9.1.3 shows the Lagrangian subproblem to be structurally blind to exactly that distinction.

2.2 Compact formulation

We adopt the compact arc-flow structure, denoted SRPS-1, of Riera-Ledesma and Salazar-González [1]. Let $V_k = J_k \cup \{0, n+1\}$ be processor k ’s node set with start depot 0 and end depot $n+1$; let $z_{i\ell}^k \in \{0, 1\}$ indicate that processor k traverses arc (i, ℓ) , $x_{j,k} \in \{0, 1\}$ that processor k serves job j , $y_j \in \{0, 1\}$ that job j is selected, and $s_{j,k}$ the service-start time of processor k at job j . The objective is $\max \sum_{j \in J} b_j y_j$ subject to route structure,

$$\begin{aligned} \sum_{\ell \in V_k \setminus \{0\}} z_{0\ell}^k &= 1, & \sum_{i \in V_k \setminus \{n+1\}} z_{i,n+1}^k &= 1, \\ \sum_{i \in V_k \setminus \{j\}} z_{ij}^k &= x_{j,k}, & \sum_{\ell \in V_k \setminus \{j\}} z_{j\ell}^k &= x_{j,k} \quad \forall j \in J_k, \end{aligned}$$

joint-service coupling (constraint 3),

$$y_j \leq x_{j,k} \quad \forall j \in J, k \in K_j,$$

and timing (constraints 1 and 2),

$$s_{\ell,k} \geq s_{i,k} + t_{i\ell} - M(1 - z_{i\ell}^k), \quad s_{j,k} = s_{j,k'} \quad \forall k, k' \in K_j, \quad s_{n+1,k} - s_{0,k} \leq L.$$

The arc-time propagation doubles as subtour elimination: start times increase along any used arc with positive transition time, so cycles are infeasible. Section 12 solves this formulation directly as an uncoupled reference.

2.3 Benchmark

We use the benchmark introduced with the problem [1]. It comprises eight families along two axes. The *distance* axis distinguishes horizontal transition costs, $|u_i - u_\ell|$, which mimic the real instrument, from Euclidean costs on the plane; families with an **E** prefix are Euclidean. The *class* axis records $|K_j|$: class-1 families have $|K_j| = 1$ for every job and reduce to the team orienteering problem, while classes 2–4 have $|K_j| \in \{2, 3, 4\}$ and carry genuine synchronisation. Profits take

values in $\{1, 10, 100\}$. Instances are labelled by family, size n , and a budget-tightness parameter $\alpha \in \{0.25, 0.50, 0.75\}$ controlling how much of the total service demand fits inside L .

Our primary study set is the 660 instances of classes 2–4, where synchronisation is active. Where a wider set of 780 appears (Section 6.4), it is the 660 plus the class-1 families, and the text says so.

3 Related work

3.1 SRPS and its exact treatment

SRPS was introduced by Riera-Ledesma and Salazar-González [1] together with the model SRPS-1 of Section 2.2, a benchmark, and an exact branch-price-and-cut algorithm in two variants: one enhancing the pricing step, one node-based. The method runs under a one-CPU-hour limit and reports a primal–dual gap of $100 \times (\text{dual} - \text{primal})/\text{primal}$.

Its reported performance defines the space in which a heuristic can add value. In the hardest class-4 families all instances up to $n \leq 55$ are solved to optimality; at $n = 60$ nonzero gaps remain, reaching 1.64% on the Euclidean family at $\alpha = 0.75$; for $n \geq 65$ no benchmark solutions are published. Two regimes therefore exist — hard instances with residual exact gaps, and larger instances with no reported exact result.

The exact method’s cut separation is the direct motivation for the first of these (Section 9.1). Branch-price-and-cut separates violated inequalities against the *fractional* solution of a linear master problem; whether that machinery transfers to a heuristic which never forms such a solution is precisely what Section 9.1 answers.

3.2 Metaheuristics for selective routing

Adaptive Large Neighbourhood Search and related destroy–repair metaheuristics are effective across orienteering, team orienteering, and vehicle-routing variants with rich side constraints, because they combine strong primal search with problem-specific insertion logic [2, 3]. A substantial body of metaheuristics has been developed for the team orienteering problem specifically [4]. These deliver strong incumbents but generally provide *no per-instance optimality certificate*, which is the gap the solver of Section 5 closes.

3.3 Lagrangian relaxation and decomposition

Lagrangian relaxation with subgradient optimisation is a classical route to instance-specific bounds: a complicating coupling is dualised, the residual problem decomposes, and the dual value bounds the optimum [5, 6, 7]. The approach has a long record in routing [9, 10], and metaheuristics interleaving such relaxations with metaheuristic search are an established way to obtain both strong primal solutions and bounds [11]. Step sizes follow Polyak [8]. Guignard’s treatment of Lagrangian *decomposition* [7] is used directly in Section 13, where the condition for decomposition to improve on relaxation — that the dualised block lack the integrality property — decides one cell of the design space without an experiment.

3.4 Techniques this study draws on

Beyond the two hypotheses, the study evaluates cutting planes and their separation discipline, Lagrangian decomposition, reduced-cost variable fixing, dual stabilisation by trust region and smoothing, parallel evaluation of independent subproblems, and direct solution of the compact model by a general-purpose MIP solver. Table 1 maps each to the lecture that introduces it.

4 Summary of techniques evaluated

Table 1 lists every technique evaluated, the metric it targets, and its outcome. **The numbered rows are the ten strengthening techniques** and are the subject of Sections 9–11. The two unnumbered rows above the first rule are not interventions: one is the solver’s own dual bound, the object the ten techniques try to strengthen, and the other is an independent reference solver used for comparison in Section 12. The unnumbered row marked with † below the final numbered row is Tomer’s positive extension, described in Section 7.

The organising observation is the split by target. Interventions aimed at the certified gap are bounded in advance by the ceiling of Table 2, and every one of them was absorbed by it; the two aimed at runtime carry no such bound, and one of them delivered. The cut-augmented extension breaks the ceiling-absorption pattern by incorporating cuts directly into the dual mechanism rather than applying them to the already-floored bound.

Table 1: Techniques evaluated in this study. Rows 1–10 are the strengthening attempts; the two rows above the first rule are the bound being strengthened and the reference solver. Row † is the cut-augmented Lagrangian extension (Section 7).

#	Technique	Target	Outcome
—	<i>Lagrangian relaxation</i>	<i>dual ub</i>	<i>the method under study (§5.5)</i>
—	<i>Compact MILP (SRPS-1) in CPLEX</i>	<i>reference</i>	<i>looser bound than the Lagrangian on 22/30 (§12)</i>
1	BPC-style cut separation	<i>dual ub</i>	falsified on three independent grounds (§9.1)
2	Dual-guided destroy / repair	primal z	+0.0037 <i>pp</i> vs. warm control at production budget (§10.1, §10.1.5)
3	Warm- μ dual effort scheduling	<i>dual ub</i>	−0.0086 <i>pp</i> at production budget (§10.1.5)
4	Primal-to-dual seeding	<i>dual ub</i>	9 tighter / 8 looser after flooring (§9.3)
5	Dual stabilisation (trust region, smoothing)	<i>dual ub</i>	budget-dependent; no transfer at the solver’s budget (§9.4)
6	Lagrangian decomposition (variable copies)	<i>dual ub</i>	provably equal to the existing bound (§13)
7	Dantzig–Wolfe reformulation / column generation	<i>dual ub</i>	master bound equals the Lagrangian dual optimum (§9.5)
8	Surrogate relaxation	<i>dual ub</i>	dominates in theory; forfeits separability (§9.5)
9	Reduced-cost variable fixing	runtime	reach 24.8% → 0.3% at 0.99 z (§11.1)
10	Parallel subproblem evaluation	runtime	1.42 × dual, bit-identical bounds (§11.2)
†	Cut-augmented Lagrangian (cap / pair / triple)	<i>dual ub</i>	30/54 wins standalone (−0.094 <i>pp</i> mean); 12/54 pipeline wins (§7)

5 The solver

An ALNS produces feasible solutions, a Lagrangian relaxation produces valid upper bounds, and an outer loop alternates between them until the gap is small enough to stop. The distinguishing feature is that the dual is not a one-off pre-processing step but a mechanism re-tightened *inside* the search, around the current incumbent.

5.1 Representation and feasibility

A solution is a selected job set and one elementary route per processor. Feasibility is maintained by the longest-path check of Section 2.1: the routes induce a precedence graph, earliest start times are longest paths within it, and the solution is feasible exactly when that graph is acyclic and

its longest path from start to end depot does not exceed L . This single check covers constraints 1 and 2 together, including synchronised idle time.

5.2 Destroy and repair operators

Three destroy operators are used. *Random destroy* removes a uniformly sampled subset of selected jobs; *worst destroy* removes the least valuable selected jobs; *Shaw destroy* removes jobs related to a random pivot under travel-based proximity. The destroy size is sampled dynamically, with escalation tiers enlarging the removed fraction on hard instances (Section 5.7).

Three repair operators are used: *profit-greedy*, *ratio*, and *regret-2*. They differ only in the order in which candidate jobs are offered to a common insertion procedure, and it is that procedure which carries the SRPS-specific content.

5.3 Synchronisation-aware insertion

Inserting a job j is not a single-route decision: every processor in K_j must accommodate it, and all must start it at the same instant. The procedure enumerates the top few local positions on each required processor route, forms their Cartesian product, and checks each cross-route placement with the longest-path test.

Proposition 1 (Insertion-complexity bound). *If at most c local candidate positions are retained per required processor for job j , the procedure evaluates at most $c^{|K_j|}$ route-position combinations, each followed by one feasibility check.*

Proof. Candidate positions are chosen independently on each processor in K_j ; their Cartesian product contains at most $\prod_{k \in K_j} c = c^{|K_j|}$ combinations, each checked once. $\square$

With $c = 3$ and class-4 jobs this is at most $3^4 = 81$ combinations per job. This combinatorial insertion is where synchronisation enters the heuristic; the rest follows a standard destroy–repair pattern.

5.4 Acceptance, adaptation, and multi-start

The search uses simulated-annealing acceptance, adaptive roulette-wheel operator selection, and exponentially smoothed score updates: worsening moves may be accepted to escape local optima, and operators producing improving or accepted moves earn larger future weight. The search cycles over six fixed random seeds.

5.5 The Lagrangian bound

The complicating constraints are the joint-service couplings $y_j \leq x_{j,k}$. Relaxing them with multipliers $\mu_{j,k} \geq 0$ gives

$$L(\mu) = \sum_{j \in J} \max\left(0, b_j - \sum_{k \in K_j} \mu_{j,k}\right) + \sum_{k \in K} O_k(\mu), \quad (1)$$

where $O_k(\mu)$ is the optimum of a single-processor orienteering problem in which job $j \in J_k$ carries modified profit $\mu_{j,k}$.

Proposition 2 (Validity of the bound). *For every $\mu \geq 0$, the SRPS optimum satisfies $z^* \leq L(\mu)$.*

Proof. Any feasible SRPS solution satisfies $y_j \leq x_{j,k}$ for all required processors, so after relaxation the nonnegative penalty terms vanish on it and its objective cannot exceed the relaxed objective at the same μ . Maximising over feasible solutions gives the claim. $\square$

Each $O_k(\mu)$ is solved *exactly* by bitmask dynamic programming in $O(2^{|J_k|}|J_k|^2)$, with no pruning, sampling, or candidate-set restriction. The exponential factor is bounded by the largest per-processor job set rather than by n : since each job requires only $|K_j| \in \{2, 3, 4\}$ of the $|K| = 55$ processors, jobs spread thinly and each processor serves few. That the resulting bound on $|J_k|$ is small, and does not grow with n , is an empirical property of the benchmark, reported with the other measurements in Section 6.2.

Two properties of $O_k(\mu)$ matter throughout. First, it is solved to optimality, so no looseness originates there. Second, it enforces route length $\leq L$ but is structurally blind to synchronisation idle time, because each processor is solved in isolation. The relaxed solution is therefore optimistic in a specific, identifiable way — a fact Section 9.1.3 exploits and Section 13 returns to.

5.6 Subgradient scheme

Multipliers are initialised by the fair profit split $\mu_{j,k} = b_j/|K_j|$, which makes the initial relaxed objective coincide with an independent-orienting bound. The dual is then minimised by projected subgradient descent with Polyak-style steps [8],

$$\eta_t = \frac{L(\mu_t) - \theta^*}{\|g_t\|^2}, \quad g_{j,k} = x_{j,k}^* - y_j^*, \quad \mu_{j,k} \leftarrow \max(0, \mu_{j,k} - \eta_t g_{j,k}),$$

clamped for stability, with a diminishing fallback schedule after 30 consecutive non-improving iterations. The level reference θ^* is the current incumbent z throughout. The best bound found is returned with its multiplier vector, retained as a warm start for the next invocation.

That the fair split is the natural *starting point* for descent, yet the one multiplier vector at which reduced profits vanish identically, is the whole of the defect analysed in Section 10.1.1.

5.7 The adaptive control loop

The destroy–repair search sits inside an outer loop allocating effort only where certification remains hard. Each *phase* runs the ALNS with cycling seeds for up to 300 seconds. If the phase improves the incumbent, the destroy *tier* resets; otherwise the method performs an in-loop Lagrangian re-bound and escalates the destroy fraction through 25%, 33%, and 40%. Three stop conditions apply: the upper bound matches the incumbent, the certified gap falls below 0.3%, or the tiers are exhausted. A per-instance cap of 3600 seconds applies throughout.

Each re-bound is capped at 200 subgradient iterations or 60 seconds, is warm-started from the current multipliers, and uses the current incumbent as the Polyak target. Because the numerator $L(\mu_t) - z$ shrinks as the bound approaches the incumbent, steps are large when the bound is loose and fine when it is nearly tight: the dual is pulled down toward the incumbent while the ALNS pushes the incumbent up, and the two chase each other. That 200-iteration budget is the configuration this study measures, and Section 9.4 shows it to be the binding constraint on the dual.

5.8 Certification

The reported certified gap is

$$\text{gap} = \frac{ub - z}{ub}, \quad ub = \lfloor L(\mu) \rfloor, \quad (2)$$

with z the incumbent. Flooring is valid because all profits are integer. A gap of $x\%$ guarantees the solution lies within $x\%$ of the optimum; a gap of 0% *proves* optimality by weak duality, and we call such instances *proven optimal*. No published benchmark value enters the algorithm: both the ALNS and the dual reference use only the current incumbent.

Equation (2) has **two levers**, and the distinction organises the study: a mechanism can improve the gap by *raising* z (primal) or by *lowering* ub (dual). Cutting planes act on the dual. Neighbourhood operators act on the primal. Sections 9 and 10 divide on exactly that line.

6 Baseline performance

6.1 Certified gaps

All runs use a single machine, six worker processes, and the configuration of Section 5.7. Across the 660-instance primary study set the mean certified gap is 0.133%, the median 0.046%, and the maximum 1.965%; 94.1% of instances are certified within 0.5% and 100% within 2%. Median runtime is 5.1 minutes, and no instance stops because of the absolute runtime cap.

6.2 Certified optima and subproblem cost

283 instances (42.9%) are returned with a certified gap of exactly zero and are therefore proven optimal in the sense of Section 5.8. Of those, 149 are also proved optimal by the exact method, where the two certificates coincide and cross-validate one another; the remaining 134 are proven optimal by the Lagrangian certificate alone.

The exactness of the dual bound rests on the per-processor job sets staying small, and they do. Across all 660 instances the largest $|J_k|$ is 13, with per-instance means of 2.8–6.0, and this maximum is *flat in n*: it is already 13 at $n = 80$ and remains 13 at $n = 150$. Each subproblem therefore costs at most $2^{13} \cdot 13^2 \approx 1.4 \times 10^6$ operations, so the bound is computed exactly even on the largest instances. Section 11.2 shows the flip side of the same distribution: because cost scales as $2^{|J_k|}$, a single processor can account for more than half an instance’s dual work.

6.3 Relation to the exact method

Against the published branch-price-and-cut results [1], the instances divide into three groups. On the 245 the exact method proves optimal, the solver recovers the optimum on 241 and is one profit unit below on four, with a mean certified gap of 0.067%. On the 112 where the exact method returns an incumbent but stops at its one-hour limit with a nonzero gap, the solver matches the published incumbent on 93, improves it on 15, and is one unit below on four. On the 300 with no reported exact solution, the solver certifies all 300, including 196 first-ever certified results.

6.4 Headroom: a ceiling on any gap-directed improvement

The results above imply a bound on what *any* technique aimed at the certified gap can achieve, and that bound governs the rest of this report.

Cuts and every other dual-side device act only on *ub*. The optimum P^* satisfies $z \leq P^* \leq ub$, since z is attained by a feasible solution. The maximum conceivable improvement from a technique driving the dual bound exactly to the optimum on every instance is therefore bounded by the current certified gap itself. The same bound applies symmetrically to primal-side techniques, which can raise z at most to P^* .

On nearly half the benchmark there is nothing to gain. On the median instance the entire theoretical prize is 0.0087 *pp*. For comparison, arm-to-arm differences in the calibration suites below range over ± 0.01 –0.09 *pp*: the median instance’s complete theoretical prize is smaller than the noise floor of the apparatus built to detect it. No amount of calibration recovers a signal beneath its own measurement noise.

This computation costs seconds and uses only data already in hand. That it was performed *after* rather than before the calibration campaigns is itself one of the lessons of Section 14.

7 Cut-augmented Lagrangian relaxation

Two cut-separation routes are closed by the later falsification study (Section 9.1): the primal incumbent is feasible by construction and can never supply a violated cut, and the natural feasibility cut against the Lagrangian relaxed solution is unsound on most instances. An alternative

Table 2: Ceiling on any gap-directed improvement, computed over 780 instances. The row set is the 660 primary instances plus the class-1 families, which reduce to team orienteering and are easier — hence a proven-optimal share of 48.5% here against 42.9% on the primary set. The maximum, 1.9650 *pp*, is identical under both row sets.

Quantity	Value
Instances already provably tight (gap = 0)	378 / 780 (48.5%)
Mean maximum possible gap reduction	0.1136 <i>pp</i>
Median maximum possible gap reduction	0.0087 <i>pp</i>
90th percentile	0.2948 <i>pp</i>
Maximum	1.9650 <i>pp</i>
Total absolute headroom $\sum(ub - z)$	2255 profit units
Share held by the top 50 instances	45.3%

route avoids both pitfalls by incorporating valid inequalities directly into the *dual* as Lagrangian penalties, so the bound-computing mechanism itself becomes tighter. This section develops and evaluates that route, independently of the falsification study.

7.1 Formulation

The Lagrangian relaxation $L(\mu)$ decomposes per-processor (Section 5.5). The cut-augmented extension $L(\mu, \gamma, \nu)$ adds two families of valid inequalities in the projected y -space, while preserving this decomposition.

7.1.1 Capacity cuts

For each processor k , let O_k denote the single-processor orienteering optimum at zero multipliers: the maximum profit processor k can earn independently. Any feasible SRPS solution selects only jobs that processor k can accommodate within the horizon L , so the total profit of selected jobs requiring k is at most O_k :

$$\sum_{j \in J_k} b_j y_j \leq O_k \quad \forall k \in K. \quad (3)$$

Introducing multipliers $\gamma_k \geq 0$ for these constraints and folding them into the Lagrangian modifies the effective profit for job j on processor k 's subproblem to $\mu_{j,k} + \gamma_k b_j / O_k$. Each per-processor subproblem remains a standard orienteering DP on the same arc graph, with augmented edge weights.

7.1.2 Incompatibility cuts

A pair (i, j) is *incompatible* if no single-processor route can accommodate both jobs simultaneously (verified by attempting to include both in any per-processor DP). For incompatible pairs:

$$y_i + y_j \leq 1. \quad (4)$$

A triple (i, j, ℓ) is *incompatible* if no route can accommodate all three:

$$y_i + y_j + y_\ell \leq 2. \quad (5)$$

Multipliers $\nu_{ij} \geq 0$ and $\nu_{ij\ell} \geq 0$ penalise violations, adding a per-job offset to the selection threshold before each per-processor solve.

Proposition 3 (Validity). *Constraints (3)–(5) are valid for SRPS.*

Proof. Any feasible SRPS solution selects only jobs every required processor can serve within L , so the selected profit requiring processor k is at most O_k , giving (3). An incompatible pair cannot both appear in any feasible route, so (4) holds by definition; similarly (5). $\square$

Proposition 4 (Decomposition preserved). $L(\mu, \gamma, \nu)$ separates into the same per-processor orienteering subproblems as $L(\mu)$, with shifted job profits.

Proof. The γ_k penalties enter linearly through $\mu_{j,k} \leftarrow \mu_{j,k} + \gamma_k b_j / O_k$, so each $O_k(\mu, \gamma)$ is still a standard orienteering DP. The incompatibility penalties shift the y -threshold before the selection rule fires, and do not enter the route subproblems. $\square$

Three configurations are evaluated: **cap** (capacity cuts only), **pair** (capacity and pair incompatibilities), and **triple** (all three). All multipliers are updated by projected subgradient steps alongside $\mu_{j,k}$, warm-started between invocations.

7.2 Experimental protocols

Three protocols probe where the cuts add value.

Standalone comparison (stratified-54). Each method receives the same total dual budget: 100 cold-start Lagrangian iterations, optionally followed by 100 warm cut-augmented iterations. The baseline LR_200_cold doubles the cold budget; the control LR_100+LR_100 applies the same 200 iterations as a warm continuation without cuts. All comparisons are gap-only: no ALNS primal is run.

Tight-UB pipeline (hard-6). Six hard instances run through the full adaptive pipeline (Section 5.7), with the initial ub set to the master-result upper bound from the production run — the tightest certificate already available. This probes whether an in-loop dual refresh can improve on it.

Headroom pipeline (stratified-54). The same 54-instance set runs through the full pipeline, with the initial ub deliberately set to $\lfloor L_{200, \text{cold}} \rfloor$ per instance — the weakened value from the standalone analysis. This gives the in-loop dual refresh genuine headroom to tighten. Two arms are compared: plain LR in-loop versus cutLR-triple in-loop.

7.3 Standalone results

Table 3: Standalone dual comparison on the stratified-54 subset (54 instances spanning 6 families, 3 α levels, 3 difficulty positions each). All methods share a total budget of at most 200 subgradient iterations. *Warm* methods begin from LR_100’s multipliers; *cold* methods start from zero. Lower certified gap is better. The pairwise comparison column counts wins against LR_200_cold, the natural double-budget baseline.

Method	Mean gap (%)	Median (%)	Max (%)	vs. LR_200_cold
LR_100 (100 cold)	1.028	0.673	4.204	—
LR_200_cold (200 cold)	0.455	0.144	2.712	baseline
LR_100+LR_100 (warm)	0.467	0.147	2.811	19W / 24L / 11T
cutLR-cap warm 100	0.397	0.116	2.351	35W / 9L / 10T
cutLR-pair warm 100	0.404	0.125	2.260	30W / 14L / 10T
cutLR-triple warm 100	0.361	0.114	2.440	30W / 13L / 11T

The three cutLR configurations improve on every aggregate metric over LR_200_cold; the triple configuration achieves the lowest mean gap (0.361%, $-0.094pp$ vs. baseline).

A decisive comparison is LR_100+LR_100 against cutLR-triple: both consume the same total dual budget (100 cold then 100 warm iterations), but the warm continuation without cuts achieves 0.467% mean gap and wins only 19 of 54 instances versus the baseline, while cutLR-triple achieves 0.361% and wins 30. The +0.106 *pp* advantage of cutLR over the warm continuation cannot be attributed to the warm start; it is the contribution of the cuts themselves.

The capacity-cut configuration (cutLR-cap) records the highest instance-win count (35), reflecting that capacity tightness is widespread across the benchmark. Adding pair incompatibilities (cutLR-pair) sharpens the per-instance bound on medium-hard instances at the cost of a marginally slower convergence per call, shifting some wins to losses on small-easy instances. The triple configuration adds incompatible triples to the pair set, recovering the lowest aggregate mean while the per-instance win count stays at 30.

7.4 Pipeline results

7.4.1 Tight-UB protocol: the ceiling applies

With the master upper bound as the initial ub , the in-loop dual refresh fires but cannot tighten the certificate: both the LR arm and the cutLR-triple arm record **zero successful refreshes** across all six hard instances. The certified gaps are identical between arms.

This is the ceiling of Section 6.4 operating directly. The initial ub was produced with a 200-iteration, 60-second dual budget; the in-loop refresh has only 100 iterations and 30 seconds. A tighter input cannot be improved by a smaller budget, and $\lfloor L_{\text{in-loop}} \rfloor \geq ub_0$ on every instance.

7.4.2 Headroom protocol

Table 4: Headroom pipeline experiment: 54 instances, initial ub set to $\lfloor L_{200,\text{cold}} \rfloor$ per instance, giving the in-loop dual refresh genuine room to tighten. LAG_MAX_ITER=100, LAG_MAX_TIME=30 s, three workers, 900 s cap. “Refreshes” counts in-loop dual calls that successfully lowered the certified bound. The *headroom-25* subset marks the 25 instances where $\lfloor \min_{\text{config}} L_{\text{cutLR}} \rfloor < ub_{\text{init}}$ (standalone cutLR provably tighter than the weakened bound).

Arm	Mean gap (%)	Median (%)	Inst. wins	UB refreshes
<i>All 54 instances</i>				
LR in-loop	0.533	0.088	8	32
cutLR-triple in-loop	0.428	0.080	12	35
<i>Headroom-25 subset</i>				
LR in-loop	1.002	0.547	—	—
cutLR-triple in-loop	0.780	0.411	—	—

Across all 54 instances the cutLR arm reduces mean certified gap by 0.105 *pp* and increases the refresh count from 32 to 35. On the 25 confirmed-headroom instances the reduction grows to 0.222 *pp* (1.002% $\rightarrow$ 0.780%); these are the cases where the standalone cutLR bound was already below the weakened initial ub , so the in-loop refresh has room to fire even within its reduced budget. The 34 ties are instances the ceiling absorbs regardless: the weakened initial ub is already close to the ALNS incumbent, or the ALNS proves optimality without a dual refresh.

Notable per-instance results. The largest cutLR win is B_n150_030_a25_088: LR records a certified $ub = 3561$ (gap 4.437%, zero refreshes) while cutLR-triple achieves $ub = 3551$ (gap 0.028%, one refresh), a difference of 10 profit units in the certified bound. The gap difference is amplified because LR’s ALNS incumbent regressed from a previous run (3550 $\rightarrow$ 3403), a primal contamination artifact of the sequential experiment design; the ub difference of 10 is the clean evidence.

The instances where plain LR wins (8 cases, all $\alpha = 0.25$ hard families) share a common pattern: accumulated warm-multiplier subgradient calls converge to a tighter final Lagrangian value than cutLR’s richer-but-slower steps reach within the 30-second refresh cap. This is the $\alpha = 0.25$ tradeoff: sparse job-processor overlap makes pair and triple incompatibilities rare, so the extra penalty terms add overhead without meaningfully cutting the relaxation.

7.5 Verdict

The cut-augmented Lagrangian demonstrably tightens the bound in standalone comparison. With the same dual budget, cutLR-triple reduces mean certified gap from 0.455% to 0.361% and wins on 30 of 54 instances; the plain warm continuation (LR_100+LR_100) achieves only 0.467% on the same budget, confirming the improvement is from the cuts rather than the warm start alone.

The pipeline result is conditional. With a tight initial *ub* the ceiling absorbs both arms and neither gains. Under the headroom protocol, cutLR-triple achieves mean 0.428% against 0.533% for plain LR, winning 12 instance comparisons to LR’s 8. The advantage is concentrated on medium-hard instances ($\alpha = 0.5$, $n \geq 80$), where per-processor capacity conflicts make subgradient LR slow to converge. On the hardest $\alpha = 0.25$ instances, accumulated warm-multiplier LR calls eventually converge past what cutLR achieves per call within the 30-second cap, giving LR a countervailing edge on 8 instances.

Two caveats apply to the headroom experiment. First, the deliberate weakening of the initial *ub* does not reflect the production pipeline, where the initial certificate comes from a prior full run; the headroom protocol is a stress-test, not a simulation. Second, the two pipeline arms share incumbents: ARM 1 (LR) updated the canonical incumbent directory, so ARM 2 (cutLR) started from those improved solutions on some instances. The *ub* comparison is unaffected by this contamination, but the gap figures on instances where ARM 1’s ALNS regressed should be read as approximate; the UB delta is the clean per-instance evidence.

8 Study design

Every technique below was implemented *outside* the solver: no file in the solver’s own modules was modified, and each instrument composes them from the outside. This keeps the comparison honest — a difference between an arm and its control cannot be an artifact of a changed baseline.

8.1 Notation

8.2 Instance subsets

- **Stratified-30.** A stratified subset across families B, C, D, EB, EC, ED and across n and α . Used wherever breadth matters.
- **Coupling-12.** A fast-screen subset for arm matrices comparing ten or more configurations.
- **Hard-tail-4.** The loosest-gap instances. Used where headroom is required for an effect to be visible at all.
- **Small-nonzero-30.** Every instance with $n \leq 50$ carrying a nonzero certified gap, for the exact reference of Section 12.

8.3 Budget regimes

Three regimes appear, and the distinction is load-bearing rather than incidental: Section 14 reports two verdicts that reverse between them.

Table 5: Notation used throughout.

Symbol	Meaning
z	incumbent objective (primal)
ub	Lagrangian upper bound, $\lfloor L(\mu) \rfloor$ (dual)
P^*	true optimum, known only to lie in $[z, ub]$
$\mu_{j,k}$	multiplier on the coupling of job j to processor k
r_j	reduced profit $b_j - \sum_{k \in K_j} \mu_{j,k}$
$g_{j,k}$	subgradient component $x_{j,k} - y_j$
G	absolute gap $L(\mu) - z$
pp	percentage points of certified gap
phase	one ALNS run of up to 300 s with cycling seeds
tier	destroy-fraction escalation level (25%, 33%, 40%)
<i>Cut-augmented Lagrangian (Section 7)</i>	
O_k	single-processor orienteering optimum for processor k at zero multipliers
γ_k	multiplier on capacity cut for processor k ; $\gamma_k \geq 0$
ν_{ij}	multiplier on pair incompatibility $y_i + y_j \leq 1$; $\nu_{ij} \geq 0$
$\nu_{ij\ell}$	multiplier on triple incompatibility $y_i + y_j + y_\ell \leq 2$; $\nu_{ij\ell} \geq 0$

Table 6: Budget regimes. *Production* is the solver’s own configuration (Section 5.7); the others are screens.

Regime	phase (s)	cap (s)	dual iters	dual (s)
Production	300	1800	200	60
Fast screen	120	240	120	20
Activation probe	20	45	—	—

The 1800 s cap is the per-instance budget of the development harness; the solver itself caps at 3600 s, and no arm reached either for reasons other than tier exhaustion.

8.4 How each technique was tested and calibrated

Cut separation (Section 9.1). An explicit cut pool with three separation routines (circuit, route-length, synchronisation-path), integrated through a hard feasibility filter and a soft scoring penalty. Calibrated by a 29-run staged suite on Stratified-30 at the production regime: a 3-run correctness gate, 12 coarse runs spanning low/medium/high settings, 8 refinement runs around the coarse finalists, and 6 robustness runs over seed bundles. Eight parameters were swept — `sep_freq_phase` $\in \{1, 2, 3\}$, `max_cuts_per_round` $\in \{5, 10, 20\}$, `epsilon_violation` $\in \{10^{-6}, 10^{-5}, 10^{-4}\}$, `epsilon_reject` $\in \{10^{-5}, 10^{-4}, 10^{-3}\}$, `lambda_cut` $\in \{0.1, 0.25, 0.5, 1.0\}$, `cut_age_limit` $\in \{2, 4, 8\}$, `cut_weight_decay` $\in \{0, 0.05, 0.1\}$, `sep_budget_s_per_phase` $\in \{5, 10, 20\}$ — with lexicographic selection on mean Δ gap, then maximum final gap, then runtime.

Activation sweep. Four points of `epsilon_violation` $\in \{10^{-7}, -50, -100, -200\}$ on a three-instance subset at the activation-probe regime, with a gate failing a run when no cut enters the pool. Negative tolerances are non-physical and were used deliberately to force activation, separating “the mechanism does not apply” from “the mechanism is broken”.

Separation against the relaxed solution. A read-only instrument runs the subgradient to convergence (200 iterations or 30 s), replays the relaxation at μ^* , and measures two violation channels. A second searches for any global ordering achieving $\text{makespan} \leq L$ by time-boxed random-restart hill climbing (90 s per instance), reporting SCHEDULABLE constructively when one is found. Neither has parameters to calibrate: they are measurements, not treatments.

Dual-guided coupling (Section 10.1). Six arms on Coupling-12 at the fast-screen regime: plain control, warm- μ control, original (v1) operators with warm μ , and corrected (v2) destroy, repair, and both. The v2 repair scores $(b_j + w r_j)/(1 + \text{cost})$ with $w = 1$; the v2 destroy scales its tie-breaking jitter to 1% of the observed spread of r_j . The decisive arm is the warm- μ control, isolating the cost of the seed from the effect of the guidance. Three arms were repeated at the production regime.

Reduced-cost fixing (Section 11.1). No search runs. The subgradient is run per instance on Stratified-30, r_j read off at μ^* , and the two fixing tests evaluated in closed form. Because the threshold G depends on incumbent quality and fixing runs mid-search rather than post hoc, reach is reported at z , $0.99z$ and $0.95z$.

Seeding and stabilisation (Sections 9.3, 9.4). Ten variants over three axes — initialisation ($\varepsilon \in \{0.05, 0.10, 0.25\}$), step rule (Polyak; trust region with radius 0.5 and 1.0), and dual smoothing (0.3, 0.6) — plus two combinations, on Stratified-30 at 200 iterations across six workers. Because a smoke test and the full sweep disagreed, a convergence probe then ran four variants to 1000 iterations on Hard-tail-4, retaining the full bound history so the running best could be read at checkpoints 60/100/200/400/700/1000 from one run per variant.

Parallel subproblem evaluation (Section 11.2). The per-processor loop is farmed to a worker pool with the instance loaded once per worker. Processors are partitioned by longest-processing-time assignment weighted by $2^{|J_k|}$; contiguous chunking was measured first and found badly imbalanced. Correctness requires the bound to be *identical*, not merely close, to the sequential path. Timing uses best-of-three repeats.

Uncoupled exact reference (Section 12). SRPS-1 is built directly and handed to CPLEX 22.1.1 with a 180 s limit and six threads, sharing no state with the solver. Only instances CPLEX proves closed admit a decomposition of the certified gap.

9 Techniques targeting the dual bound

Everything here aims to lower *ub*, and is therefore bounded by Table 2 before any of it is run.

9.1 BPC-style cut separation

9.1.1 First design and its structural impossibility

The initial implementation maintained a pool of cuts of three types — circuit, route-length, and synchronisation-path — separated at phase boundaries against the ALNS incumbent, applied through a hard feasibility filter and a soft scoring penalty.

This design cannot work, for a reason requiring no experiment. Separation is performed against the incumbent, and the incumbent is feasible by construction. Feasibility means precisely: no cycles in the constraint graph, no route over L , and a synchronisation path within L (Section 2.1). These are exactly the three conditions the separator tests. The candidate set is therefore empty at every call, identically, on every instance.

A 29-run calibration suite was nevertheless executed on Stratified-30, consuming 127.9 minutes and tuning eight parameters. Its outcome was a tautology: the recommended configuration was the arm with all cut machinery disabled, and the mean objective delta was identical (-3.300) across the top five arms. Every run reported `cuts_added_total` = 0.

9.1.2 The activation sweep and mechanism inertness

To separate “the mechanism is broken” from “the mechanism does not apply”, a four-point sweep drove the violation tolerance ε negative, forcing cuts into the pool.

Table 7: Activation sweep on a three-instance subset. Pool size responds to ε ; nothing else does.

ε	cuts added	mean viol. after	filter rejections	mean Δ gap (pp)	final objectives
10^{-7}	0	0.0	0	+0.057444	4971 / 2285 / 2226
-50	2	0.0	0	+0.057444	4971 / 2285 / 2226
-100	8	0.0	0	+0.057444	4971 / 2285 / 2226
-200	15	0.0	0	+0.057444	4971 / 2285 / 2226

The pool grows fifteenfold; every outcome column is bit-identical to six decimal places. That is not a statistical null — a real treatment with no effect produces scatter — but an algebraic identity, and it has a specific cause. The evaluation routine scores each cut against a right-hand side frozen at construction as the true horizon L :

$$\text{viol} = \max(0, \text{max_len} - c.\text{rhs}), \quad c.\text{rhs} = L.$$

For any feasible candidate $\text{max_len} \leq L$, so the clamp returns exactly 0.0 regardless of ε . The guard `if viol > 0.0` never fires, no cut is ever recorded as violated, the hard filter never rejects, and the soft penalty subtracts $\lambda \cdot 0$. Fifteen active cuts produced `mean_violation_after` = 0.0.

We call this *mechanism inertness*: the machinery activates and reports activation while remaining incapable of influencing the outcome variable. It is more dangerous than an obvious crash, because it produces clean-looking null results that invite the interpretation “the idea does not help” when the correct interpretation is “the idea was never tested”.

A telemetry defect compounded it. A counter intended to record penalty applications was incremented once per job removal regardless of any active cut, reporting nonzero values on runs with an empty pool. An analyst reading only the summary columns would have seen a mechanism apparently doing work.

9.1.3 Redirecting separation at the relaxed solution

The structural argument also indicates the repair. In branch-price-and-cut, separation runs against a *fractional* solution, which is where violations live. The heuristic has no such solution, but it does have the Lagrangian relaxed solution, and that object is infeasible in a well-defined way: $g_{j,k}$ is nonzero until convergence, and each $O_k(\mu)$ ignores synchronisation idle time (Section 5.5).

We built a read-only instrument (S0) that runs the subgradient to convergence, replays the relaxation at μ^* , and measures two channels:

Channel A (consistency). Jobs partially claimed — served by some but not all processors in K_j , violating constraint 3 directly.

Channel B (synchronisation time). The joint difference-constraint system over the relaxed routes: whether it is cyclic, and if not, whether its longest path exceeds L .

Both fire abundantly. Notably, the same cycle-detection routine that returned zero on every one of the 29 calibration runs returns nonzero here *without modification*. The cut family was never wrong; the object it was pointed at was.

Table 8 summarises S0 over the 30-instance stratified subset. Both violation channels fire on essentially the whole set.

Table 8: S0 violation instrument over 30 instances. Every per-processor route respects the horizon, yet the joint structure does not.

Quantity	Value
Channel A: partially claimed jobs	184 (184 on multi-processor jobs)
Mean subgradient norm $\ g\ ^2$ at μ^*	72.5
Channel B: instances cyclic under per-processor optimal order	21 / 30
Total circuits detected	148
Acyclic instances with makespan $> L$	4 / 9
mean / max excess over L	1401.0 / 2044.0
Every per-processor route within L	yes

The contrast in the last two rows is the substantive finding. The decomposition certifies every individual route as fitting inside the horizon, while the joint difference-constraint system does not admit a schedule under those same routes. The per-machine subproblem cannot represent the constraint that binds, because that constraint is exactly the one the relaxation dualised.

9.1.4 Why the natural cut is unsound

Channel B appears to be the opportunity: structural infeasibility the decomposition cannot see. It does not survive scrutiny.

The relaxation fixes *which* jobs each processor takes, not the *order*. Any single global total order induces per-processor sequences whose arcs all point forward, hence an acyclic graph. Acyclicity is therefore always achievable, and the cycles S0 reports are an artifact of each processor adopting its own locally optimal min-time order. The real question is whether some ordering also satisfies makespan $\leq L$.

This is a soundness requirement, not a technicality. A no-good cut asserting “this relaxed selection is infeasible” is valid only if *no* ordering works. If a feasible ordering exists and the cut is applied anyway, a feasible point is excluded, *ub* becomes invalid, and every certified gap computed downstream is wrong. For a method whose contribution is certification, that failure mode is worse than adding no cuts at all.

Instrument S0b therefore searches for any global ordering achieving makespan $\leq L$ by time-boxed random-restart hill climbing, reporting SCHEDULABLE when one is found.

S0b searches for any global ordering achieving makespan $\leq L$. A SCHEDULABLE verdict is constructive: an explicit feasible ordering is exhibited, so the selection provably cannot be cut.

Table 9: S0b schedulability over 30 instances. A schedulable selection cannot be excluded by a no-good cut without invalidating the bound.

Outcome	Instances
SCHEDULABLE (feasible ordering found)	16 / 30 (53.3%)
None found (inconclusive, not a proof)	14 / 30 (46.7%)

On 16 of 30 instances the relaxed selection is provably schedulable, so a cut asserting its infeasibility would exclude a feasible point and invalidate every downstream certificate. The margins are narrow: the tightest feasible schedule clears the horizon by 1 time unit, and 12 of the 16 schedulable instances clear it by less than 50.

That narrowness compounds the problem. A separation routine would have to distinguish “feasible by a handful of time units” from “infeasible”, and in a certifying method a misclassification does not produce a weak cut: it produces an invalid upper bound and a silently wrong certified gap. The standard discipline for user cuts is explicit on this point—a valid inequality must strengthen the relaxation *without excluding feasible integer points* [13, 12]—and this cut family cannot meet it.

9.1.5 Verdict

We falsify cut separation on three independent grounds. First, separation against the incumbent cannot produce a violated cut, confirmed operationally by a 29-run suite reporting no cuts in any run. Second, forcing activation admitted up to fifteen cuts while leaving mean post-evaluation violation at 0.0 and every final objective bit-identical: the cuts were *inert* rather than ineffective, and an experiment with no power cannot support a claim about efficacy. Third, redirecting separation at the relaxed solution yields a cut family that is *unsound* on 16 of 30 instances. Independently of design, Table 2 caps any cut family at 0.1136 *pp* mean and 0.0087 *pp* median.

Three limits should be stated. We do not claim BPC cuts are ineffective for SRPS in general: we tested three families inside one architecture. The 14 of 30 instances on which no feasible ordering was found remain *formally open*, since the schedulability search is heuristic; the soundness argument rests only on the 16 constructive cases, which suffices, because a single wrongly excluded feasible point invalidates the bound. And of the three limbs, only the ceiling generalises beyond the tested subsets.

9.2 Can the incumbent itself generate cuts?

A natural question, worth answering explicitly because it motivates much of the design above: the solver always holds a best known solution, so is there information in it that can be used to generate cuts?

The answer is no, and the reason is Section 9.1.1 stated in general form. A cut is useful only if some point violates it. The incumbent is feasible, so it violates no valid inequality — that is

what feasibility means. Any separation routine run against it returns an empty candidate set, whatever the cut family, independently of the particular families we implemented.

What the incumbent *can* supply is a bound, not a cut. It is a lower bound on the optimum, and the solver already exploits that: z is the Polyak level reference (Section 5.6), the threshold in the fixing tests of Section 11.1, and one end of the interval producing the ceiling of Table 2. The distinction is between a feasible point, which bounds, and an infeasible one, which cuts. Generating cuts requires an infeasible object, and in this architecture the only one available is the Lagrangian relaxed solution — which is why Section 9.1.3 redirects separation there, and Section 9.1.4 shows why even that fails.

9.3 Primal-to-dual seeding and dual stabilisation

The solver seeds μ by the fair split and chains warm starts from one re-bound to the next; the incumbent enters only as the Polyak target, never as structure (Section 5.6). Seeding μ from the incumbent’s own selection is a genuine primal-to-dual coupling and was untested. Dual stabilisation — trust region and smoothing — was likewise untested against the plain Polyak scheme.

Two coupling directions from the syllabus remained untested after the dual-guided operators: primal-to-dual seeding, in which the incumbent’s own selection sets the starting multipliers rather than only serving as the Polyak target; and dual stabilisation via trust region or smoothing [14]. Both are evaluated below over 30 instances at 200 subgradient iterations.

Table 10: Dual variants over 30 instances, 200 iterations. ΔUB is the change in the real-valued bound against baseline; negative is tighter. The final column is what survives $\lfloor \cdot \rfloor$, which is what the certified gap actually uses.

Variant	mean ΔUB	wins / 30	floor: tighter / looser
baseline (fair split, Polyak)	+0.000	0	—
incumbent seed, $\varepsilon = 0.05$	-0.415	14	6 / 8
incumbent seed, $\varepsilon = 0.10$	-0.811	15	9 / 8
incumbent seed, $\varepsilon = 0.25$	-0.858	12	6 / 8
trust region, radius 0.5	+12.478	14	7 / 14
trust region, radius 1.0	+1.576	18	9 / 10
dual smoothing, 0.3	+17.328	5	1 / 23
dual smoothing, 0.6	+53.899	0	0 / 29
seed 0.10 + trust 0.5	+12.616	13	6 / 14
seed 0.10 + smoothing 0.3	+13.855	8	4 / 19

Seeding produces a small mean tightening (-0.81 for $\varepsilon = 0.10$, winning on 15 instances of 30), but the median instance moves by -0.000 —that is, not at all. The mean is carried by 10 instances with $|\Delta\text{UB}| \geq 1$, spanning -10.0 to $+3.0$. After flooring, the direction is a coin flip, and two of the three seeding variants are net worse. There is no usable improvement here.

Smoothing is straightforwardly harmful and monotone in its weight. The trust region requires more care, and is treated separately below, because its verdict depends on the iteration budget.

9.4 Does the trust region survive a longer budget?

At 60 iterations the trust region appeared decisively ahead (-8.07 mean). At 200 it was behind ($+1.58$). That sign reversal is the signature of a starved baseline, the same failure this study documents for the screening budget, so the question had to be settled at a budget where the baseline is allowed to finish. Each variant was therefore run once at 1000 iterations with its bound history retained, and the running best read off at checkpoints.

Table 11: Bound trajectories to 1000 subgradient iterations. **prod** is the bound the production configuration reports, which uses at most 200 iterations warm-started.

Instance prod	Variant	60	100	200	400	700	1000
ED_n050_011_a50_032	baseline	1308.00	1307.45	1306.70	1306.30	1306.30	1306.30
1307							
	trust05	1319.10	1311.08	1306.15	1306.01	1306.00	1306.00
1307							
	trust10	1309.37	1307.12	1306.28	1306.07	1306.05	1306.05
1307							
	seed_eps10	1308.00	1307.81	1307.16	1306.31	1306.31	1306.31
1307							
EB_n110_006_a50_017	baseline	3334.37	3315.45	3283.32	3257.70	3254.17	3253.54
3259							
	trust05	3339.00	3332.44	3320.78	3300.94	3275.49	3254.66
3259							
	trust10	3335.55	3327.42	3305.58	3271.27	3253.84	3253.10
3259							
	seed_eps10	3347.50	3327.50	3273.31	3256.03	3254.12	3253.46
3259							
B_n140_021_a25_061	baseline	3314.00	3272.00	3234.77	3199.73	3194.34	3192.64
3195							
	trust05	3365.35	3335.34	3289.67	3228.13	3204.08	3190.54
3195							
	trust10	3333.36	3295.92	3233.06	3203.97	3188.54	3187.46
3195							
	seed_eps10	3313.70	3270.75	3230.83	3197.30	3193.42	3191.90
3195							
D_n080_041_a50_122	baseline	2243.01	2242.34	2240.95	2238.51	2236.60	2235.94
2235							
	trust05	2238.84	2237.82	2236.42	2235.11	2234.58	2234.46
2235							
	trust10	2241.62	2239.68	2237.71	2236.26	2235.15	2234.81
2235							
	seed_eps10	2243.11	2242.07	2240.21	2237.32	2235.97	2235.40
2235							

At 1000 iterations the trust region leads on all four instances, so the 200-iteration verdict was premature rather than the 60-iteration one being spurious: the variants had simply not separated yet. After flooring, 2 of the four show a strictly tighter integer bound (B_n140_021_a25_061 by 5, D_n080_041_a50_122 by 1).

Two effects must be separated before claiming anything, the same decomposition that exposed the warm- μ confound. Part of the gain against production is simply running the subgradient longer: baseline alone improves several instances once given 1000 iterations rather than 200. Only the residual is attributable to stabilisation. On B_n140 the bound moves from 3195 (production) to 3192 (baseline at 1000) to 3187 (trust region at 1000), so of eight integer units, five come from the budget and three from the trust region.

The practical conclusion is budget-conditional and therefore does not transfer to the solver as configured. It caps the dual at 200 iterations or 60 seconds, warm-started and invoked only when a phase fails to improve—and at 200 iterations the trust region is worse on average and 9-to-10 after flooring. Its advantage exists only at roughly five times that budget, on the hardest instances, at a proportionate runtime cost. We therefore report it as a scoped observation about the dual in isolation, not as a recommended change.

9.5 Alternative ways to combine the subproblem solutions

Equation (1) combines the per-processor optima by plain summation: $L(\mu)$ is the y -term plus $\sum_k O_k(\mu)$. That is the only aggregation the solver performs, and it is worth asking what alternatives exist and what they would buy. Three are available, and all three can be settled without an experiment.

Lagrangian decomposition. Duplicate the shared variables and price their disagreement rather than summing independent blocks. For the natural split this provably reproduces the same bound, because the dualised block has the integrality property; Section 13 gives the argument. A different split, separating routing from synchronisation timing, is not covered by it and remains the one open direction.

Column generation and Dantzig–Wolfe. Reformulate with one column per per-processor route and price columns by the same orienteering subproblem. By the Dantzig–Wolfe equivalence, the linear master’s bound at convergence equals the Lagrangian dual optimum $\min_{\mu} L(\mu)$ for the same subproblem structure. Column generation therefore cannot yield a *tighter* bound than a fully converged subgradient; it can only reach it differently. That is not nothing — an LP master does not oscillate, and Section 9.4 shows the subgradient still descending when its budget expires — but the improvement on offer is one of convergence, not of bound quality, and it is bounded by the same ceiling.

Surrogate relaxation. Aggregate the coupling constraints into a single weighted surrogate rather than dualising each. The surrogate bound dominates the Lagrangian bound in theory, but the surrogate subproblem retains the aggregated constraint and is no longer separable by processor — forfeiting precisely the property that makes $O_k(\mu)$ exactly solvable. The aggregation that would tighten the bound is the one that destroys the decomposition.

The pattern across all three matches the ceiling result: the aggregation is not where the looseness lives. The looseness lives in the subproblem’s blindness to synchronisation waiting (Section 5.5), and no way of combining blind subproblem solutions restores what the blindness discarded.

9.6 Warm-start dual effort scheduling

The isolating control introduced for the dual-guided operators (Section 10.1) doubles as a test of a dual-side technique in its own right: spending a subgradient solve *before* the search rather than only reactively when a phase stalls. Its result is reported with those arms, because the two are measured together, and Section 10.1.5 shows it to be a screening artifact.

10 Techniques targeting the primal incumbent

10.1 Dual-guided destroy and repair

10.1.1 Design and the initialisation degeneracy

Two operators were added to the ALNS portfolio. A guided destroy removes jobs ranked by ascending reduced profit r_j , breaking ties on descending dual pressure. A guided repair inserts jobs by a score derived from r_j and the insertion cost. Multipliers are optionally refreshed from the in-loop re-bound.

Multipliers are initialised by the fair profit split (Section 5.6). Then

$$\sum_{k \in K_j} \mu_{j,k} = |K_j| \cdot \frac{b_j}{|K_j|} = b_j \quad \implies \quad r_j = b_j - \sum_{k \in K_j} \mu_{j,k} = 0$$

for *every* job, identically. Direct measurement confirms it: on one 80-job instance, 0 of 80 jobs have nonzero reduced profit; on a 150-job instance, 0 of 150.

The fair split is the correct starting point for descent, but it is precisely the multiplier vector at which the guidance signal vanishes. Two consequences follow mechanically:

Destroy degenerates to random removal. The sort key is $(0 + \text{uniform}(-10^{-6}, 10^{-6}), -\text{dual pressure})$.

The first component is pure noise on an identically zero base, and exact ties have measure zero, so the intended second criterion never breaks anything.

Repair degenerates to insertion order. With $r_j = 0$ every job takes a constant fallback score, the strict improvement test fails for every candidate after the first, and insertion follows the iteration order of the unserved set.

Both degenerate operators are inserted at position 0 of their portfolios, displacing tuned operators.

10.1.2 The original results are what the defect predicts

Table 12: Original dual-guided matrix (30 instances). Lower mean Δgap is better.

Arm	mean Δgap (pp)	mean Δobj	Predicted by
baseline	0.000313	−0.033	—
full	0.003921	−0.067	both defects, partly rescued by feedback
destroy only	0.008313	−0.200	random destroy displacing a tuned operator
repair only	0.016217	−0.367	insertion-order repair displacing regret-2

The ordering is not noise: it is what the degeneracy analysis predicts in advance. The repair-only arm, displacing regret-2 with order-dependent insertion, is worst; the destroy-only arm, displacing a tuned operator with randomness, is next; the baseline wins.

10.1.3 Correcting the design

Testing the hypothesis rather than the defect requires two repairs. **Warm multiplier initialisation** seeds μ from a real subgradient solve: nonzero reduced profits rise from 0/80 to 74/80, with spread 9.667. **Non-degenerate operators (v2)** let the dual signal *modulate* a competent base heuristic instead of replacing it, so that with $r \equiv 0$ the repair reduces to profit-density greedy rather than to a constant. Both original operators are retained behind flags, so the degenerate behaviour remains reproducible.

10.1.4 Corrected results

Table 13 reports the corrected matrix, in which the dual signal is live at the point of use (Section 10.1.3).

Table 13: Corrected dual-guided matrix. Lower mean Δgap is better. The δ column is measured against the warm- μ control where available, so the seed cost is not charged to the guidance.

Arm	n	mean Δgap (pp)	mean Δobj	δ vs. base (pp)
control (no dual operators, no warm μ)	12	0.1663	-0.250	+0.1310
control + warm μ (isolates seed cost)	12	0.0353	-0.250	—
v1 operators + warm μ	12	0.0559	-0.750	+0.0206
v2 destroy only + warm μ	12	0.0316	-0.417	-0.0037
v2 repair only + warm μ	12	0.0478	-0.667	+0.0125
v2 destroy + repair + warm μ	12	0.0324	-0.333	-0.0030

The matrix decomposes cleanly once the warm- μ control is included. Warm initialisation alone accounts for -0.1310 pp of the movement; the guided operators, measured against that same baseline, span -0.0037 pp to +0.0206 pp, and 2 of the 4 are worse than the warm control outright.

Two columns settle the attribution. The mean objective delta is identical for the plain and warm controls (-0.250 in both), so warm initialisation changed nothing in the primal—its entire contribution is a tighter *ub*. The guided operators, by contrast, degrade the objective (to -0.750) while moving the gap by at most 0.0037 pp in the favourable direction, inside the ± 0.01 –0.09 pp arm-to-arm noise band of the earlier suites and below the 0.0087 pp median ceiling of Table 2.

The naive reading of this matrix—comparing guided arms against the *plain* control—reports a 0.1347 pp improvement and concludes that corrected dual guidance works. It does not. That apparent effect is the warm- μ solve, and it would have been reported as a positive result had the isolating arm not been run. We regard this as the most instructive artifact of the study: a confound that survives every check except the one that varies the suspected cause on its own.

The hypothesis is therefore falsified as stated, while the experiment yields a positive finding of its own: dual effort spent *before* the search, rather than reactively when a phase fails to improve, tightens the certificate materially on this subset. This is a scheduling question about the dual, not a steering question about the primal, and it belongs to the dual-stabilisation family [14] rather than to guidance. Section 15 records the budget caveat that qualifies it.

10.1.5 Production-budget validation

The fast screen runs at `abs_cap` = 240s and `lag_max_time` = 20s, against production defaults of 1800s and 60s—13% of the runtime and 33% of the dual budget. Since the effect isolated above is a *dual* effect, the screen may simply have starved the baseline. We therefore re-ran the decisive arms at full budget.

The warm- μ effect measures -0.0086 ppat production budget, against -0.1310 pp in the screen—a loss of 93% of its magnitude. The mechanism is visible in the control itself: its mean final gap falls from 0.4239% under the screen to 0.2636% here. The screen cut instances off before the

Table 14: Production-budget validation on the same 12 instances. The warm- μ effect does not survive a properly-fed dual.

Arm	n	mean Δgap (pp)	mean Δobj	mean final gap (%)
control (no dual operators, no warm μ)	12	0.0061	-0.083	0.2636
control + warm μ	12	-0.0026	+0.083	0.2550
v2 destroy + repair + warm μ	12	0.0012	+0.000	0.2587

dual converged, so an extra upfront subgradient solve supplied effort the baseline was otherwise denied. Given a full budget, the existing in-loop re-bounding supplies that effort anyway, and the advantage disappears.

Guidance measured against the warm control at production budget is +0.0037 pp—still nothing, and still on the wrong side of zero.

This is the study’s second false-signal mode, and the more dangerous one in practice. Reduced-budget screening is standard for parameter exploration, and here it produced a confident, plausible, and entirely spurious 0.13 pp result. Unlike the inert-cut null, nothing internal to the screen betrays it: the arms are consistent, the ordering is stable, and the effect is far outside run-to-run noise. Only re-running the finalist at full budget exposes it.

11 Techniques targeting runtime

Runtime carries no ceiling. Both techniques here were selected for that reason.

11.1 Reduced-cost variable fixing

Lagrangian reduced-cost fixing is the one syllabus technique whose target is runtime rather than gap, and runtime carries no ceiling. Writing $G = L(\mu) - z$, forcing a single y_j shifts the bound by one term only:

$$L(\mu \mid y_j=1) = L(\mu) + \min(0, r_j), \quad L(\mu \mid y_j=0) = L(\mu) - \max(0, r_j),$$

so $r_j < -G$ proves j absent from every optimal solution and $r_j > G$ proves it present. Both tests are $O(1)$ per job given r_j , which the dual already computes.

Table 15: Reach of reduced-cost fixing over 30 instances, as a function of incumbent quality. Fixing runs mid-search, not post-hoc.

Incumbent used for G	Mean jobs fixable
final z (post-hoc, near-optimal)	24.8%
0.99 z	0.3%
0.95 z	0.0%
Overall (final z)	481 / 2490 (19.3%)
Median per instance	0.7%
Instances with any fixable job	15 / 30

The rule has no practical reach. Backing the incumbent off by one per cent collapses the mean from 24.8% to 0.3%, and by five per cent to 0.0%. The median instance yields 0.7% even post-hoc.

The reason is instructive, and it inverts an argument that looked compelling beforehand. We expected an asymmetry: the same tightness that caps cut-based strengthening should make the fixing threshold G easy to clear. It does—observed G runs as low as 0.0000—but a small G does

not create fixing opportunity. It reports that the instance is already closed, at which point there is no remaining search to accelerate. Both properties are consequences of the same quantity, so they cannot be played against each other.

11.2 Parallel subproblem evaluation

Profiling attributes 84% of dual wall time to `orienteering_dp_with_selection`, invoked once per processor per subgradient iteration. The per-processor solves are independent by construction [5] — that independence is what the relaxation buys — so the loop parallelises. Unlike every other intervention tested here, this one cannot alter the objective, the bound, or validity—only wall-clock—so it cannot be falsified in the same way.

Table 16: Parallel dual, 4 instances, 4 workers, 200 iterations, best of 3 repeats. Timing spread across repeats averaged 1.4%.

Instance	sequential (s)	parallel (s)	speedup	% of ceiling
B_n140_021_a25_061	14.36	10.13	1.42×	75%
EB_n110_006_a50_017	6.53	6.07	1.08×	72%
D_n080_041_a50_122	14.46	10.09	1.43×	44%
ED_n050_011_a50_032	0.47	0.27	1.73×	43%
mean			1.42×	59%

Bounds were identical to the sequential path in every configuration and every repeat, which is the correctness claim the approach needs.

Two structural limits are worth recording. First, DP cost is $O(2^m m^2)$ in a processor’s candidate count m , and m ranges from 0 to 13 across the 55 processors, so cost is extremely skewed: on B_n140 a single processor carries 53% of the instance’s total dual work. Because no partition splits one DP, speedup is capped at total/max_single regardless of worker count—a mean ceiling of 2.66×

 here. Second, contiguous chunking is badly wrong for such a distribution (observed imbalance up to 5.35× at eight workers); longest-processing-time assignment recovers most of the difference, lifting one instance from 1.13× to 1.71×.

The end-to-end value is nonetheless small, because the component is not on the critical path. In the production-budget arms the dual was invoked on one instance of twelve; the others terminated on the gap threshold before any phase failed to improve. On the single instance that did use it, the dual accounted for roughly 45 of 1297 seconds, or 3.5% of runtime, so a 1.42×

 component speedup yields about 1.01× overall and exactly 1.00× elsewhere. The honest summary is a correct and measured component improvement whose component does not matter much in the current architecture; it would matter in a cold-start regime where the initial bound must be computed rather than loaded.

12 An uncoupled exact reference

The ceiling argument bounds any strengthening by $(ub - z)/ub$ because the optimum P^* is known only to lie in $[z, ub]$. Solving instances exactly would pin P^* down and convert that bound into a measurement. We therefore solved the compact SRPS-1 arc-flow model directly in CPLEX 22.1.1—standalone, sharing no state with the ALNS or the Lagrangian—on the 30 instances with $n \leq 50$ that still carry a nonzero certified gap. Instances already certified optimal are excluded: their decomposition is trivially zero.

CPLEX closed 7 of 30 within a 180-second limit. Only those admit a valid decomposition; on a timed-out run CPLEX’s incumbent is not P^* , and on 10 of the 23 unproven instances it was strictly worse than the ALNS incumbent.

Table 17: The 7 instances CPLEX closed. *looseness* is $(ub - P^*)/ub$, recoverable by a tighter dual; *subopt* is $(P^* - z)/ub$, which no dual improvement can reach.

Instance	P^*	z	ub	looseness	subopt
ED_n040_002_a75_006	1026	1026	1029	0.29%	0.00%
ED_n040_004_a50_011	1164	1154	1165	0.09%	0.86%
D_n045_007_a25_019	662	662	663	0.15%	0.00%
C_n050_004_a50_011	2017	2016	2017	0.00%	0.05%
EC_n050_004_a50_011	1885	1885	1886	0.05%	0.00%
C_n050_004_a25_010	1242	1242	1243	0.08%	0.00%
D_n050_012_a25_034	753	753	754	0.13%	0.00%

The ALNS is *provably optimal* on 5 of these 7, so on those the entire certified gap is bound looseness and nothing is left for the primal to recover. Across the unproven instances CPLEX matched the ALNS on 12 and beat it on 1, which is external verification of incumbent quality from a solver sharing no code with the heuristic.

The more useful finding concerns the bound. On 22 of 30 instances the Lagrangian certificate is *strictly tighter* than the dual bound CPLEX reaches within its limit. The theory anticipates this: the Lagrangian subproblem is an orienteering problem solved exactly by dynamic programming, and orienteering lacks the integrality property, so its Lagrangian bound may strictly dominate the LP relaxation of the compact model. It does here. This is also why the literature attacks SRPS by branch-price-and-cut rather than solving SRPS-1 directly: the compact relaxation is weak.

The decomposition the experiment was built for is nonetheless limited, for a structural reason worth recording. A direct MILP reaches only small instances, and small instances are largely closed already—75% of those with $n \leq 50$ have a certified gap of exactly zero—leaving 30 candidates of which CPLEX closed 7. The instances where the split would matter carry gaps above 1.5% at $n = 110$ –150, far beyond reach when the specialised branch-price-and-cut of the literature closes only $n \leq 60$. On the hard tail the ceiling remains an analytic bound; it cannot be replaced by measurement.

13 Lagrangian decomposition: why the natural split cannot help

One cell of the design space calls for Lagrangian *decomposition* rather than the Lagrangian *relaxation* the solver implements. We did not run it, because for the natural split it can be shown not to help.

Write the problem as $\max\{c^\top v : v \in X_A \cap X_B\}$, where X_A collects the per-processor routing and horizon constraints and X_B the joint-service coupling $x_{j,k} = y_j$, both intersected with integrality. The solver dualises X_B and solves over X_A exactly, so its bound is

$$z_{\text{LR}} = \max\{c^\top v : v \in \text{conv}(X_A), x_{j,k} = y_j\}.$$

Lagrangian decomposition duplicates the shared variables, assigns one copy to each block, and prices their disagreement:

$$z_{\text{LD}} = \max\{c^\top v : v \in \text{conv}(X_A) \cap \text{conv}(X_B)\}.$$

By the Guignard–Kim result [7], $z_{\text{LD}} \leq z_{\text{LR}}$ always, with strict improvement exactly when $\text{conv}(X_B)$ is strictly contained in the linear description of X_B — that is, when the dualised block lacks the integrality property.

Here it does not lack it. X_B is the set of binary vectors satisfying $x_{j,k} = y_j$: equalities tying each $x_{j,k}$ to a single binary y_j . Fixing y determines x , and the remaining polytope is a product

of $\{0,1\}$ -valued coordinates, so every vertex of the linear description has $y_j \in \{0,1\}$. Hence $\text{conv}(X_B)$ coincides with that description, and

$$z_{\text{LD}} = z_{\text{LR}}.$$

Duplicating variables for the coupling constraint is guaranteed to reproduce the bound the solver already computes. No experiment is required, and none would be informative: the two bounds are equal by construction, so any measured difference could only be numerical noise.

The split that could help. A different partition is not covered by this argument. Separating *routing* from *synchronisation timing* — one block enforcing per-processor routes, the other the difference-constraint system on start times — dualises a copy constraint whose blocks both have non-integral hulls, so $z_{\text{LD}} < z_{\text{LR}}$ becomes possible. This is also the split targeting a deficiency we measured rather than a hypothetical one: the per-processor subproblem enforces route length but is blind to synchronisation waiting (Sections 2.1, 5.5).

We record it as future work. It requires a second subproblem family, a timing oracle, and a second multiplier vector, and its payoff remains capped by Table 2. It is the only remaining idea in this study with a mechanism pointing at a measured source of looseness, which is why we name it precisely rather than leaving the cell blank.

14 Methodological lessons

Activation is not effect. A mechanism can fire, report firing, and still be incapable of changing the outcome. The activation sweep of Table 7 passed an explicit activation gate at $\varepsilon = -100$ while remaining completely inert. Gates must therefore be placed on the *outcome* variable: before spending a calibration suite, assert on a small subset that the treated arm differs from control. That guard would have caught the failure of Section 9.1 in roughly fifteen minutes instead of 128.

Budgets decide verdicts, in both directions. Reduced-budget screening is standard practice, and it produced here a confident, internally consistent, entirely spurious 0.13 *pp* result. Nothing inside the screen betrays it: arm ordering is stable and the effect sits far outside run-to-run noise. The failure mode is specific — a starved baseline flatters any treatment that supplies the starved resource.

The same mechanism runs the other way, and we walked into it. The trust region measured -8.07 at 60 iterations, $+1.58$ at 200, and led on every instance at 1000. We declared it falsified on the middle number. It was not falsified; the arms had not yet separated. A negative verdict taken at a single budget is as unsafe as a positive one. Three of this study’s findings — the fast screen, the trust region, and reduced-cost fixing — turn on this, which suggests it is the dominant confound in metaheuristic calibration rather than an incidental one.

Check the ceiling before calibrating. Table 2 costs seconds and uses only data already in hand. It establishes that the median instance offers 0.0087 *pp* of headroom — below experimental noise — which is decisive information about whether a calibration campaign can possibly succeed. It should precede the campaign, not follow it.

Verify that the signal is non-degenerate. The defect of Section 10.1.1 is a single arithmetic identity, $\sum_k b_j / |K_j| = b_j$, checkable in one line before any experiment. Guidance mechanisms should assert that their signal has nonzero variance across the decision set *at the point of use*.

Distinguish soundness failures from performance failures. The result of Section 9.1.4 is not that the cut performs poorly, but that it is *invalid* on a majority of instances. In a certifying method, an invalid cut does not degrade the result — it corrupts the guarantee, silently. Soundness must be established before efficacy is measured.

15 Threats to validity

The schedulability search of Section 9.1.4 is heuristic. A SCHEDULABLE verdict is constructive and therefore sound: an explicit feasible ordering is exhibited. A “none found” verdict is not a proof of infeasibility, and we do not treat it as one; those instances remain formally open, and settling them would require an exact feasibility oracle over the disjunctive orderings, which is NP-hard in general.

The S0 and S0b samples are subsets of the benchmark rather than its entirety, selected to include both the stratified subset and the loosest-gap instances. The ceiling argument carries no such limitation, being computed over 780 instances, and it is the argument on which the principal conclusion rests.

The corrected dual-guided arms carry a runtime confound by construction: warm initialisation spends an additional subgradient solve the plain control does not pay. We control for it with a dedicated arm applying warm initialisation with both guided operators disabled. A second confound, the reduced screening budget, is not merely controlled but *measured*: Section 10.1.5 re-runs the decisive arms at full budget.

All results are measured against the solver configured as in Section 5.7, in particular a dual refresh capped at 200 iterations or 60 seconds. Section 9.4 shows that budget to be the binding constraint on the dual: the subgradient is still descending when it stops. Raising it tightens the certificate on every hard-tail instance tested, by 0.29 *pp* on average at essentially no runtime cost. Conclusions about gap-directed techniques become *more* secure under such a configuration, since a tighter *ub* shrinks the ceiling and leaves any technique less room; conclusions that are themselves budget-conditional — the trust region above all — would need re-testing.

Finally, we tested three specific cut families and one dual-guidance design. We do not claim that no cut or coupling mechanism can help SRPS heuristics in general. We claim that these do not, that the reasons are structural, and that the ceiling bounds what any successor could achieve on this benchmark.

16 Conclusions

We described a certified primal–dual solver for SRPS, tried ten times over to strengthen it with techniques from the exact-algorithms syllabus (nine of which failed on structural grounds), and then exhibited a positive dual-side contribution: a cut-augmented Lagrangian relaxation that demonstrably tightens the bound when the initial certificate has genuine headroom.

Falsification study: nine of ten attempts fail.

Cut separation fails three times over (Section 9.1). Separation against a feasible incumbent is mathematically incapable of finding a violated cut. Redirected at the Lagrangian relaxed solution it finds violations in abundance, but the natural feasibility cut is unsound on the majority of instances tested, because the relaxed selection admits a feasible synchronised schedule once ordering is chosen globally rather than per processor. And independently of design, the certified bound is already exact on 48.5% of instances, capping every cut family at 0.11 *pp* mean and 0.009 *pp* median improvement.

Dual-guided coupling fails twice over. The operator design is degenerate under its own initialisation, and correcting that defect was worth doing because it converts an untested hypothesis into a tested one. The corrected matrix then appears positive, and is not: an isolating control attributes the movement to the warm-start dual solve rather than to guidance, and a

full-budget re-run shows that residue to be a screening artifact as well. Guidance measured against a properly-fed control is $+0.0037\text{ pp}$ — on the wrong side of zero.

Of the remaining techniques, Lagrangian decomposition of the natural split is provably equivalent to the relaxation already in use; seeding and stabilisation fail to move the floored bound; and reduced-cost fixing, though sound and cheap, fires only once the incumbent is already essentially optimal, at which point no search remains to accelerate. The expected asymmetry — that the tightness capping cut-based strengthening should make the fixing threshold easy to clear — does not hold, because both properties are consequences of the same quantity G and cannot be played against each other.

Parallelising the per-processor subproblems is the one intervention that improves what it targets: $1.42\times$ on the dual with bit-identical bounds, capturing 59% of a ceiling itself limited because a single subproblem can carry half an instance’s work. Its end-to-end value is nonetheless about one per cent, because the dual is invoked on one instance in twelve. A correct optimisation of something that is not the bottleneck is still not a contribution to the solver, and we report it as such.

The cut-augmented Lagrangian is the one positive dual-side result (Section 7). Capacity and incompatibility inequalities in the projected y -space, combined via Lagrangian penalties while preserving the per-processor decomposition, tighten the certified bound on 30 of 54 stratified instances in standalone comparison (0.361% mean versus 0.455% for matched-budget plain LR, -0.094 pp). In a pipeline experiment with deliberately weakened initial bounds the cut-augmented arm wins 12 of 54 instance comparisons against 8 for plain LR, with advantage concentrated on medium-hard instances. The benefit is conditional: where the initial certificate is already tight the ceiling absorbs both arms equally, and on the hardest low-alpha instances accumulated warm-multiplier subgradient calls eventually outpace per-call cutLR. The net picture is that cuts add real value at the start of a solve or after a change in instance data, but not inside a re-solve whose initial bound is already the output of a full production run.

The broader contribution is diagnostic. Mechanism inertness — machinery that activates, reports activation, and cannot affect the outcome — is a failure mode producing clean, publishable-looking null results from experiments that never took place. The guard against it is cheap and general: gate on the outcome variable, never on whether the mechanism fired. And no verdict, positive or negative, should be believed at a single computational budget.

A Reproduction and results

All instruments are read-only with respect to the solver and modify no existing code path. The accompanying repository ships a single runner:

```
python reproduce.py --list          # every stage, its command, its runtime
python reproduce.py --all          # everything, about 6 hours
python reproduce.py --all --quick  # everything at reduced budgets
python reproduce.py --fragments    # rebuild this paper’s tables from the CSVs
```

Every table in Sections 9–12 is generated directly from the result files rather than transcribed. Artifacts are written to `results/bpc_replica_dev/` (cut separation and its instruments), `results/dual_guided_dev/` (coupling, seeding, stabilisation, fixing, parallel), and `results/exact/` (the CPLEX reference). Result files are timestamped and never overwritten, so repeated runs accumulate.

The per-instance baseline results — incumbent, bound, certified gap, and runtime for all 780 instances — are in `results/adaptive_master.csv`. Every Δ reported in this paper is measured against that table, so any arm can be recompared against the baseline directly.

The 71 instance files the experiments touch are bundled with the repository under CC0; the full benchmark is not needed to reproduce anything reported here. Repository documentation

covers setup, the meaning of every result column, and the integer-floor convention of Equation (2), which is the single most common source of misreading: a technique can improve the real-valued bound and change the certified gap by nothing.

References

- [1] J. Riera-Ledesma, J.-J. Salazar-González, Selective routing problem with synchronization, *Computers & Operations Research* 135 (2021) 105465. doi:[10.1016/j.cor.2021.105465](https://doi.org/10.1016/j.cor.2021.105465).
- [2] S. Ropke, D. Pisinger, An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows, *Transportation Science* 40 (4) (2006) 455–472. doi:[10.1287/trsc.1050.0135](https://doi.org/10.1287/trsc.1050.0135).
- [3] P. Vansteenwegen, W. Souffriau, D. Van Oudheusden, The orienteering problem: A survey, *European Journal of Operational Research* 209 (1) (2011) 1–10. doi:[10.1016/j.ejor.2010.03.045](https://doi.org/10.1016/j.ejor.2010.03.045).
- [4] C. Archetti, A. Hertz, M. G. Speranza, Metaheuristics for the team orienteering problem, *Journal of Heuristics* 13 (1) (2007) 49–76. doi:[10.1007/s10732-006-9004-0](https://doi.org/10.1007/s10732-006-9004-0).
- [5] M. L. Fisher, The Lagrangian relaxation method for solving integer programming problems, *Management Science* 27 (1) (1981) 1–18. doi:[10.1287/mnsc.27.1.1](https://doi.org/10.1287/mnsc.27.1.1).
- [6] M. Held, P. Wolfe, H. P. Crowder, Validation of subgradient optimization, *Mathematical Programming* 6 (1) (1974) 62–88. doi:[10.1007/BF01580223](https://doi.org/10.1007/BF01580223).
- [7] M. Guignard, Lagrangean relaxation, *Top* 11 (2) (2003) 151–200. doi:[10.1007/bf02579036](https://doi.org/10.1007/bf02579036).
- [8] B. T. Polyak, Minimization of unsmooth functionals, *USSR Computational Mathematics and Mathematical Physics* 9 (3) (1969) 14–29. doi:[10.1016/0041-5553\(69\)90061-5](https://doi.org/10.1016/0041-5553(69)90061-5).
- [9] M. L. Fisher, K. O. Jørnsten, O. B. G. Madsen, An optimization algorithm for the vehicle routing problem with time window constraints, *Operations Research* 45 (3) (1997) 395–406. doi:[10.1287/opre.45.3.395](https://doi.org/10.1287/opre.45.3.395).
- [10] N. Kohl, J. Desrosiers, O. B. G. Madsen, M. M. Solomon, F. Soumis, 2-path cuts for the vehicle routing problem with time windows, *Transportation Science* 33 (1) (1999) 101–116. doi:[10.1287/trsc.33.1.101](https://doi.org/10.1287/trsc.33.1.101).
- [11] C. Archetti, M. G. Speranza, A survey on matheuristics for routing problems, *EURO Journal on Computational Optimization* 2 (4) (2014) 223–246. doi:[10.1007/s13675-014-0030-7](https://doi.org/10.1007/s13675-014-0030-7).
- [12] C. Barnhart, E. L. Johnson, G. L. Nemhauser, M. W. P. Savelsbergh, P. H. Vance, Branch-and-price: Column generation for solving huge integer programs, *Operations Research* 46 (3) (1998) 316–329. doi:[10.1287/opre.46.3.316](https://doi.org/10.1287/opre.46.3.316).
- [13] H. Crowder, E. L. Johnson, M. W. Padberg, Solving large-scale zero-one linear programming problems, *Operations Research* 31 (5) (1983) 803–834. doi:[10.1287/opre.31.5.803](https://doi.org/10.1287/opre.31.5.803).
- [14] O. du Merle, D. Villeneuve, J. Desrosiers, P. Hansen, Stabilized column generation, *Discrete Mathematics* 194 (1–3) (1999) 229–237. doi:[10.1016/S0012-365X\(98\)00213-1](https://doi.org/10.1016/S0012-365X(98)00213-1).